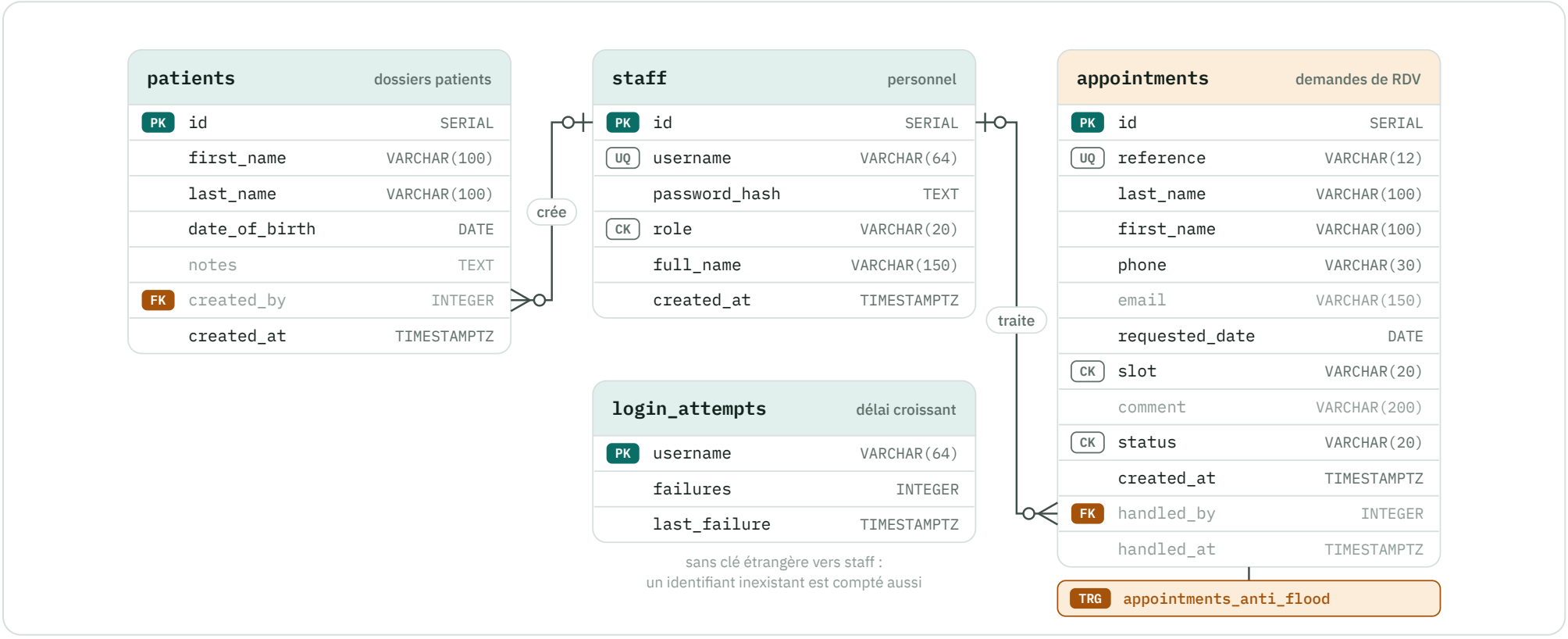
PostgreSQL 16 · VM db-medical · 192.168.50.2 · VLAN 50

Base medical_office

Projet final Jedha, profil Medical Office · mis à jour le 11 octobre 2026

Quatre tables, trois rôles et un trigger. Plus un service est exposé au public, moins son compte a de droits dans la base : le site public, ouvert à tous, peut seulement ajouter des demandes de rendez-vous.

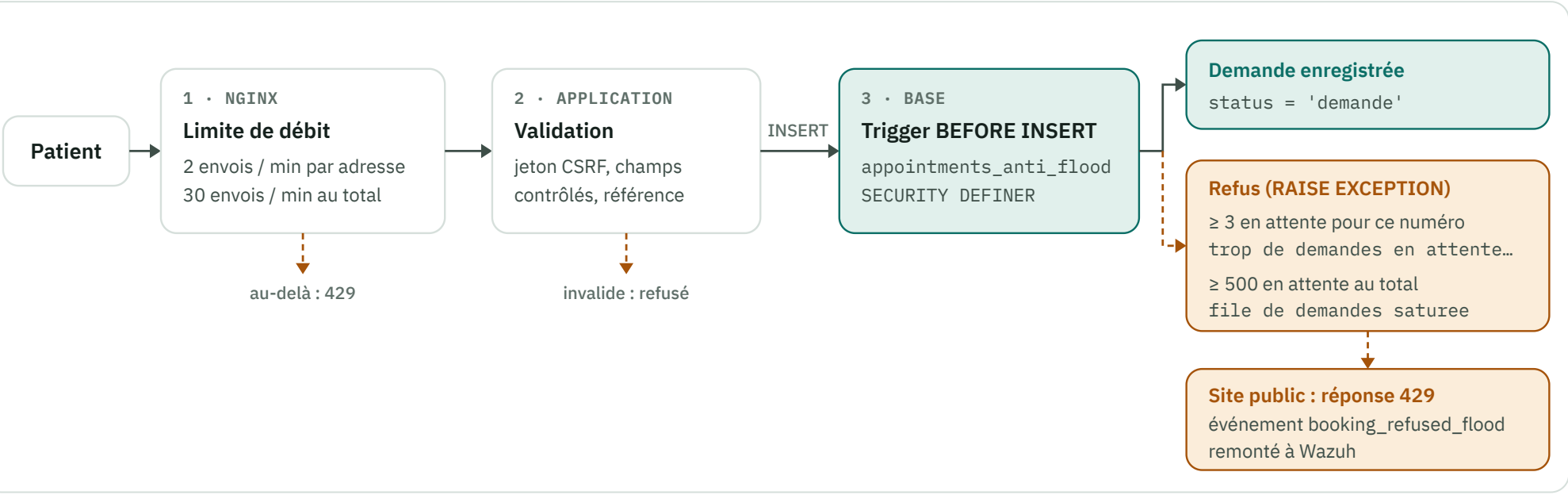
Tables et relations



PK clé primaire · FK clé étrangère vers `staff.id` · UQ valeur unique · CK valeurs contraintes par `CHECK` · `colonne` facultative, `NULL` autorisé · table interne · table alimentée par le site public · TRG trigger exécuté avant chaque insertion

Ce qui protège la table `appointments`

Une demande du site public traverse trois barrières avant d'être enregistrée. Un attaquant qui volerait les identifiants de `booking_user`, par exemple en piratant la VM de la DMZ, pourrait se connecter directement à la base et éviter nginx et l'application. Il n'éviterait pas le trigger : il vit dans la base et s'applique à toute insertion.



Pour compter les demandes, le trigger doit lire la table, ce que `booking_user` n'a pas le droit de faire. La fonction s'exécute donc avec les droits de son propriétaire (`SECURITY DEFINER`), sans qu'on ait à donner ce droit à `booking_user`. Les numéros sont comparés chiffres seuls : `06 11 22 33 44` et `0611223344` sont le même patient. Sans verrou, deux demandes envoyées au même instant compteraient chacune sans voir l'autre et pourraient dépasser le seuil (race condition) ; le verrou `pg_advisory_xact_lock` les fait passer une par une.

Une fonction aux droits élevés ne doit pas pouvoir être détournée. Quand elle cite la table `appointments`, PostgreSQL la cherche dans une liste de schémas, le `search_path`, qui dépend normalement de la session de l'appelant. Un appelant capable de placer en tête de cette liste un schéma à lui, contenant sa propre table `appointments`, ferait travailler la fonction sur son objet, avec les droits du propriétaire. La fonction fixe donc son `search_path` à `public` : elle utilise toujours les vraies tables, quel que soit l'appelant. Ici, aucun rôle applicatif ne peut créer d'objets, mais la règle s'applique par principe. Enfin, le droit d'exécuter la fonction est retiré à tous (`REVOKE ALL … FROM PUBLIC`) : elle ne sert qu'au trigger.

Qui peut faire quoi

Droits accordés par `GRANT` à chacun des rôles. Le site public ne peut qu'insérer des demandes : il ne lit aucune table, pas même celle qu'il alimente. Le portail ne supprime jamais une demande : il la confirme ou la refuse.

RÔLE	staff	patients	appointments	login_attempts
postgres_admin Administration · propriétaire des tables et du trigger	**S** **I** **U** **D**	**S** **I** **U** **D**	**S** **I** **U** **D**	**S** **I** **U** **D**
app_user Portail du personnel · web-medical	**S** **I** **U** **D**	**S** **I** **U** **D**	**S** I **U** D	**S** **I** **U** **D**
booking_user Site public · dmz-rdv	S I U D	S I U D	S **I** U D	S I U D

S SELECT, lecture · **I** INSERT, création · **U** UPDATE, modification · **D** DELETE, suppression · case colorée : droit accordé ; case grise : refusé

D'où l'on peut se connecter

`pg_hba.conf` est la liste des connexions que PostgreSQL accepte : quel rôle, vers quelle base, depuis quelle adresse, et comment il prouve son identité. Tout ce qui n'y figure pas est refusé, par exemple `booking_user` depuis le portail, ou `postgres_admin` depuis le réseau. `scram-sha-256` : mot de passe vérifié sans jamais circuler ; `trust` : aucun mot de passe demandé.

TYPE	RÔLE	BASE	DEPUIS	MÉTHODE
host	app_user	medical_office	192.168.50.3/32 · web-medical	`scram-sha-256`
host	booking_user	medical_office	192.168.60.2/32 · dmz-rdv	`scram-sha-256`
host	tous	toutes	127.0.0.1, ::1 · dans le conteneur	`scram-sha-256`
local	tous	toutes	socket Unix · dans le conteneur	`trust`

Avant même d'arriver à PostgreSQL, une connexion vers le port 5432 est filtrée par pfSense (pour la DMZ), puis par le pare-feu de la VM base (chaîne `DOCKER-USER`), qui ne laisse passer que le portail (`192.168.50.3`) et le site public (`192.168.60.2`). La dernière ligne, `local … trust`, vient de l'image officielle : elle sert aux commandes d'administration lancées à l'intérieur du conteneur (dump, purge, `psql`), ce qui exige déjà un shell sur la VM base et les droits `sudo`.

Choix de conception

Aucun lien entre demandes et dossiers

Une demande de rendez-vous n'est pas un dossier patient. Le site public ne référence jamais un patient existant ; le rapprochement se fait par le secrétariat, au téléphone.

Écrire sans pouvoir relire

Le site public est la seule partie exposée à Internet : s'il est compromis, son compte ne doit rien pouvoir divulguer. `booking_user` peut donc ajouter une demande (`INSERT`), mais pas lire la table (`SELECT`) : même avec ses identifiants, impossible de consulter les demandes des autres patients, ni même celle qu'on vient d'envoyer. Conséquence : l'application ne peut pas récupérer l'identifiant créé par la base, car `INSERT … RETURNING` est une lecture. Elle tire donc elle-même la référence au hasard avant l'insertion, et la contrainte `UNIQUE` garantit qu'elle n'est pas déjà prise.

Aucun droit sur la structure

Les tables appartiennent à `postgres_admin`, et PostgreSQL 16 interdit par défaut de créer des objets dans le schéma `public`. Ni `app_user` ni `booking_user` ne peuvent créer, modifier ou supprimer une table : même une injection réussie ne pourrait pas exécuter un `DROP TABLE`.

Rôles du personnel gérés par l'application

Le médecin et la secrétaire passent par le même compte de base, `app_user`. C'est le portail qui réserve la suppression d'un dossier au médecin : la base ne distingue pas les deux rôles, d'où le droit `DELETE` d'`app_user` sur `patients`.

Vocabulaire contraint

`role` vaut `medecin` ou `secretaire`, `slot` vaut `matin` ou `apres-midi`, `status` vaut `demande`, `confirme` ou `refuse`, avec `demande` par défaut. Une valeur hors de ces listes est refusée par la base elle-même, même si l'application avait un défaut.

Traçabilité

Chaque ligne porte un `created_at` rempli par la base. Une demande traitée enregistre `handled_by` et `handled_at` : on sait qui l'a confirmée ou refusée, et quand.

Une décision est définitive

Le portail ne met à jour une demande que si elle est encore en attente (`WHERE status = 'demande'`) : une demande confirmée ne peut plus être refusée, ni l'inverse, et `handled_by` ne peut pas être réécrit. Si deux secrétaires traitent la même demande au même moment, seule la première décision compte.

Mots de passe du personnel

`staff.password_hash` contient un hachage bcrypt de coût 12, jamais le mot de passe lui-même. Si la table était volée, retrouver un mot de passe obligerait à le deviner, et chaque essai est volontairement lent.

Délai croissant par identifiant

`login_attempts` compte les échecs de connexion au portail. À partir du cinquième, une attente s'impose : 15 s, puis 30 s, 1 min, et ainsi de suite jusqu'à 15 min ; le compteur repart de zéro après une connexion réussie ou une heure sans échec. Aucun compte n'est verrouillé définitivement.

Pas de clé étrangère vers `staff`

`login_attempts.username` est le nom saisi, pas `staff.id` : les identifiants inexistants sont comptés de la même façon, et le portail vérifie toujours un hachage. Le temps de réponse ne révèle donc pas quels comptes existent.

L'anti-flood vit dans la base

Le trigger s'applique à toute insertion, même faite directement avec les identifiants de `booking_user`. Quand il refuse, la base renvoie une erreur (`RAISE EXCEPTION`) ; le site public la reconnaît, affiche un message clair au patient (code 429) et écrit un événement `booking_refused_flood`, que Wazuh peut transformer en alerte.

Conservation limitée

Une demande est supprimée six mois après sa création, qu'elle soit en attente, confirmée ou refusée. Le formulaire l'annonce au patient. Un minuteur systemd (`purge-demandes.timer`) lance la purge chaque nuit à 3 h, ou au démarrage suivant si la VM était éteinte.

Index

En plus des index automatiques des clés primaires et des contraintes `UNIQUE` (par exemple `login_attempts.username` et `appointments.reference`) :

INDEX	PORTE SUR	SERT À
appointments_status_idx	`appointments (status, requested_date)`	afficher les trois listes du portail (en attente, confirmées, refusées), déjà triées par date souhaitée ; compter les demandes en attente (tableau de bord, et éventuellement le plafond de 500 du trigger)
appointments_phone_pending_idx	`appointments (regexp_replace(phone, '\D', '', 'g'))` `WHERE status = 'demande'`	rendre rapide le comptage du trigger, sur les chiffres seuls du numéro. Partiel : il ne contient que les demandes en attente, les seules que le trigger compte

Limites assumées

LIMITE	POURQUOI ELLE EST ACCEPTÉE
À l'intérieur du conteneur de la base, n'importe quel compte se connecte sans mot de passe (ligne `local … trust`)	Y entrer exige déjà un shell sur la VM base et les droits `sudo` : à ce niveau, l'attaquant peut de toute façon copier les fichiers de la base
La liaison entre les applications et la base n'est pas chiffrée : requêtes et données circulent en clair sur le réseau interne	Pour les intercepter, il faudrait déjà contrôler le portail, la base ou pfSense, par où passe le seul flux de la DMZ. Les mots de passe ne circulent jamais (SCRAM). Activer TLS dans PostgreSQL (`ssl = on`, `sslmode=require`) est l'amélioration suivante
Les notes médicales sont stockées en clair : fichiers de la base, sauvegardes, export du projet GNS3	En fonctionnement, les droits d'accès les protègent (réseau, rôles, `sudo`). Les chiffrer (`pgcrypto`, ou chiffrement du disque) protégerait aussi une copie volée : c'est l'amélioration suivante
Quelqu'un qui envoie 500 demandes avec des numéros différents bloque le formulaire pour tout le monde	Le plafond empêche la table et la charge du secrétariat de grossir sans fin ; le formulaire redevient disponible dès que des demandes sont traitées
Un attaquant qui prendrait le contrôle du portail pourrait modifier un compte du personnel, par exemple le mot de passe de `dr.martin` (`app_user` peut écrire dans `staff`)	Ce droit sert à `seed.py` et à la rotation des mots de passe ; à ce stade, l'attaquant lit déjà tous les dossiers patients
Le champ de précisions (`comment`) peut contenir une information médicale, si le patient passe outre l'avertissement	Il sert aux contraintes pratiques (accès, horaires) ; 200 caractères au plus, lu par le seul personnel, supprimé après six mois. Filtrer le contenu automatiquement serait peu fiable
La traçabilité des décisions ne dure que six mois	La purge supprime aussi `handled_by` et `handled_at` ; au-delà, seuls les journaux envoyés à Wazuh en gardent la trace
Si la VM de la base est perdue, ses sauvegardes le sont aussi	Dump automatique chaque nuit et restauration testée ; la seule copie hors de la VM est l'export du projet GNS3